



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

OlympusX
May 2026

Prepared by
ZanyBonzy
Varun_Sharma
radev_eth

Introduction

A time-boxed security review of the **OlympusX** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A code review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About OlympusX

OlympusX is a DeFi yield protocol on Ethereum built around a deflationary ERC20 token (OLY) with an 8B fixed supply. The protocol launches through a 45-day mint phase with three vesting schedules, dynamic daily pricing (2% increment), and a referral bonus system. Post-launch, it generates yield through three channels: Lido ETH staking, Uniswap V3/V4 liquidity farming, and automated buy-and-burn. All protocol revenue flows through TaxRouter, which splits incoming ETH across five destinations - stakers, buy-and-burn, validators, farms, and genesis - at configurable BPS ratios. Trading operates through a custom Uniswap V4 hook that enforces dynamic sell taxes across four market-cap tiers (8%-38%), manages on-chain limit orders with epoch-based fill tracking, and captures all LP swap fees for the protocol.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

review commit hash - [53a70a27c84c12ae981d3a59512205cc94763b13](#)

Scope

The following smart contracts were in scope of the audit:

- `src/constants/DefaultValues.sol`
- `src/interfaces/external/AggregatorV3Interface.sol`
- `src/interfaces/external/ILido.sol`
- `src/interfaces/external/ILidoLocator.sol`
- `src/interfaces/external/IStETH.sol`
- `src/interfaces/external/ISwapRouter02.sol`
- `src/interfaces/external/IWithdrawalQueue.sol`
- `src/interfaces/external/IWstETH.sol`
- `src/interfaces/IFarmKeeper.sol`
- `src/interfaces/ILaunchManager.sol`
- `src/interfaces/IOlympusXAuction.sol`
- `src/interfaces/ISTakingVault.sol`
- `src/interfaces/ITruncGeoOracle.sol`
- `src/interfaces/IWETH9.sol`
- `src/libs/FarmKeeperLib.sol`
- `src/libs/FarmKeeperUniLib.sol`
- `src/libs/FarmKeeperV3Lib.sol`
- `src/libs/FarmKeeperV4Lib.sol`
- `src/libs/FarmKeeperViewLib.sol`
- `src/libs/Farms.sol`
- `src/libs/GracePeriodPenalties.sol`
- `src/libs/MarketCapLib.sol`
- `src/libs/OlympusXHookLib.sol`
- `src/libs/StakingCalculations.sol`
- `src/libs/SwapActionsLib.sol`
- `src/utils/Common.sol`
- `src/EthPrice.sol`
- `src/FarmKeeper.sol`
- `src/LaunchManager.sol`
- `src/OlympusX.sol`
- `src/OlympusXAuction.sol`
- `src/OlympusXHook.sol`
- `src/StakingVault.sol`
- `src/SwapActions.sol`
- `src/TaxRouter.sol`
- `src/ValidatorVault.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 3 issues
- Medium: 4 issues
- Low and Informational: 26 issues

Findings Summary

ID	Title	Severity	Status
[H-01]	Market-sell dynamic tax is hard-coded to TIER1 (38%) on exact-output sells, over-charging users	High	Fixed
[H-02]	M-03 Fix For Feb 2026 Audit Reintroduces H-03 From Dec 2025 Audit	High	Fixed
[H-03]	<code>referralPoolAllocation</code> can be drained by anyone and can cause tokens to remain stuck in the launch manager	High	Fixed
[M-01]	Dec 2025 M-07 fix is bypassable — 1 wei deposit per cycle permanently freezes the auction	Medium	Fixed
[M-02]	Vault migration permanently locks all unclaimed inflation rewards	Medium	Fixed
[M-03]	Limit-order sell tax is bypassable via <code>kill()</code> and arbitrageable via fixed 4% rate	Medium	Acknowledged
[M-04]	<code>setDistributionBps</code> doesn't validate mint path - underflow bricks entire mint phase	Medium	Fixed
[L-01]	Swagger UI Falls Back to Hardcoded Credentials in Non-Production	Low	Fixed
[L-02]	Fix L-25 only partially applied: <code>removeRewardToken()</code> was never implemented despite being marked as resolved	Low	Fixed
[L-03]	<code>handlePlaceAfter</code> sends the hook's entire ETH balance to the <code>place()</code> caller	Low	Fixed
[L-04]	Anyone can earn large rewards by staking just 1 wei Olympus token after zero stakers period	Low	Acknowledged
[L-05]	No functionality to fully withdraw the staked eth	Low	Fixed
[L-06]	New observation is written even when fees are collected from the pools	Low	Fixed
[L-07]	Claiming rewards for dead address is possible	Low	Acknowledged
[L-08]	TaxRouter all-or-nothing distribution - single broken recipient bricks all protocol ETH flow	Low	Fixed
[L-09]	Max stakes per user limit missing - unbounded <code>userStakeIds</code> growth	Low	Fixed
[L-10]	<code>setWethToken(false)</code> desyncs from cached <code>isNativePool</code> - TaxRouter cascade revert	Low	Fixed
[L-11]	<code>stakeETH</code> wraps donated stETH via full balance read - phantom harvest rewards	Low	Fixed

ID	Title	Severity	Status
[L-12]	<code>_prepareWithdrawalAmounts</code> split produces sub-minimum amounts - harvest DoS	Low	Fixed
[L-13]	<code>harvest()</code> bare call to Lido WithdrawalQueue blocks all staker rewards	Low	Fixed
[I-01]	<code>StakingVault._withdrawTokens</code> Silently Caps ETH Withdrawals to Available Balance	Informational	Fixed
[I-02]	<code>LaunchManager.mint</code> self-referral check is trivially bypassed via sybils	Informational	Acknowledged
[I-03]	Redundant zero <code>_owner</code> address check	Informational	Fixed
[I-04]	Calling <code>extendMintPhase</code> before <code>startMintPhase</code> permanently bricks launch	Informational	Fixed
[I-05]	<code>createAndInitializeLiquidityPool</code> is permissionless with oracle-dependent minting	Informational	Acknowledged
[I-06]	Grace period effective window is ~8 days, not the documented 7	Informational	Fixed
[I-07]	<code>MarketCapLib.getTokenPriceInETH</code> has misleading function name and comments	Informational	Fixed
[I-08]	Duplicate <code>ETHReceived</code> event emission in <code>_claimFeesFromPoolManager</code>	Informational	Fixed
[I-09]	Multiple critical admin setters across five contracts emit no events	Informational	Acknowledged
[I-10]	<code>FarmKeeperLib.validateSwapSlippage</code> allows 100% slippage	Informational	Acknowledged
[I-11]	<code>FarmKeeper.setLiquiditySlippage</code> natspec does not match code	Informational	Fixed
[I-12]	Permissionless <code>claimRewardsForUser</code> enables reward timing manipulation	Informational	Acknowledged
[I-13]	Dead code in <code>_executeClaim</code> wastes gas on unused external reads	Informational	Fixed

Detailed Findings

[H-01] Market-sell dynamic tax is hard-coded to TIER1 (38%) on exact-output sells, over-charging users

Severity

Impact:

Medium

Likelihood:

High

Description

In OlympusXHook.sol, `_beforeSwap` handles the sell-tax path for exact-output swaps (`params.amountSpecified > 0`):

```
uint256 netRequested = uint256(int256(params.amountSpecified));
uint256 taxBps = _safeGetCurrentMarketSellTaxBps(0); // ← passes 0 as
ethPriceUSD
uint256 taxAmount = (netRequested * taxBps) / (BPS - taxBps);
...
poolManager.take(key.currency0, taxRouter, taxAmount);
```

`_safeGetCurrentMarketSellTaxBps(0)` eventually calls `src/libs/MarketCapLib.sol:148-150`:

```
if (oldestObservationSecondsAgo < twapPeriod || _ethPriceUSD == 0) {
    return TIER1_TAX_BPS; // 3800 = 38%
}
```

Because the hook hard-codes `_ethPriceUSD = 0` in the exact-output path, `MarketCapLib` skips the market-cap calculation and always returns the highest tier (38%) whenever `TaxMode.Dynamic` is configured.

In contrast, the exact-input path in `_afterSwap` / `_applySellTax` (`src/OlympusXHook.sol:604-605` and `src/OlympusXHook.sol:670`) correctly fetches the real ETH price with `_safeGetEthPriceUSD()` and forwards it, so it obtains the correct dynamic tier (8% / 18% / 28% / 38% depending on market cap).

Every user that performs an exact-output sell of OLY → ETH while the hook is in `Dynamic` mode and the market cap is at Tier 2 or higher is forced to pay the maximum 38% tax rather than the 28% / 18% / 8% tier they should have paid for that market cap. For a protocol whose market cap is expected to grow past \$25M/\$100M/\$1B, this is a permanent, silent over-tax that transfers user principal to the `taxRouter`. The loss is deterministic (difference between `TIER1_TAX_BPS` and the true tier) and applies to every routing path (aggregators, periphery routers, contracts) that uses exact-output, which is a standard swap mode on V4.

Additionally, in `src/OlympusXHook.sol:614` the `emitSwapEvent` call passes `_safeGetCurrentMarketSellTaxBps(0)` (again 0) for the event's `taxBps` field even for exact-input swaps, so the emitted `OlympusXSwap.taxBps` is also wrong and inconsistent with the actual

`taxAmount` that was charged in `_applySellTax`. Off-chain consumers (UI, accounting, analytics, vesting / unlock calculators) that read `taxBps` from this event will mis-report the tax rate for every market sell.

Recommendations

Compute and forward the real ETH price in both the exact-output path and the event emission so the same pricing inputs are used across `_beforeSwap`, `_afterSwap`, and the event.

[H-02] M-03 Fix For Feb 2026 Audit Reintroduces H-03 From Dec 2025 Audit

Severity

Impact:

High

Likelihood:

Medium

Description

- **H-03** (Dec 2025 audit): Identified that limit orders are incorrectly filled when the price enters and reverses out of a range. Recommended partial-fill accounting.
- **M-03** (Feb 2026 audit): Found that the team's H-03 fix (adding `continue` guards) caused a different bug — it permanently skipped orders at the starting boundary tick during downward price drops, even when those orders were legitimately fully crossed.

The team resolved M-03 by removing the `continue` guards entirely, which fixed M-03 but re-introduced H-03. The current code has no protection against the bug — the guards are gone and partial-fill accounting was never implemented.

Step-by-Step Reproduction

Setup: `tickSpacing = 200`, limit sell order at range `[0, 200]` with `zeroForOne = false`, initial `tickLowerLast = -200`.

Swap 1: Tick moves from `-200` → `100` (`params.zeroForOne = false`, selling OLY for ETH)

1. `_getCrossedTicks`: `tickLower = _getTickLower(100, 200) = 0`, `tickLowerLast = -200`
2. `tickLower (0) >= tickLowerLast (-200)` → else branch → `lower = -200`, `upper = 0 - 200 = -200`
3. Loop: `_fillEpoch(key, -200, true)` — fills the order at tick `-200` only
4. `tickLowerLast` set to `0`

Swap 2: Tick moves from `100` → `-100` (`params.zeroForOne = true`, buying OLY with ETH)

1. `_getCrossedTicks: tickLower = _getTickLower(-100, 200) = -200, tickLowerLast = 0`
2. `tickLower (-200) < tickLowerLast (0) → if branch → lower = -200 + 200 = 0, upper = 0`
3. Loop: `_fillEpoch(key, 0, false)` — **fills the [0, 200] sell order**
4. But the price only went from tick 100 to -100 — it **never crossed tick 200** (the upper boundary of the [0, 200] range)

The [0, 200] sell order is marked as fully filled despite the price never crossing its upper boundary. When the user withdraws, they receive the current position composition — predominantly the OLY they deposited, not the ETH they expected.

- A user's limit sell order is incorrectly marked as filled, giving them back mostly their own deposited tokens instead of the desired output token.
- The order is permanently removed from the pool (`_setEpoch` resets to `EPOCH_DEFAULT`), so the user cannot wait for a proper fill later.
- The `filled` flag is boolean with no partial-fill bookkeeping — users have no way to distinguish a legitimate fill from this incorrect one.

The core issue is that `_getCrossedTicks` produces a range of tick buckets that were traversed during a swap, but the system treats every bucket in that range as fully crossed. A proper fix requires distinguishing between:

1. **Fully crossed ranges** — price entered through one boundary and exited through the other. These should be filled.
2. **Partially entered ranges** — price entered through one boundary but reversed back through the same boundary. These should NOT be filled (or should receive partial-fill accounting).

Recommendations

Replace the current unconditional fill logic with direction-aware boundary checks. A limit order at range `[tickLower, tickLower + tickSpacing]` should only be filled if the price **crossed both boundaries** — meaning it entered through one side and exited through the other.

[H-03] `referralPoolAllocation` can be drained by anyone and can cause tokens to remain stuck in the launch manager

Severity

Impact:

High

Likelihood:

High

Description

By setting referrer as the launch manager address the referral pool allocation will be decreased but the tokens will remain in the contract and will deny the future legitimate referrers and minters from their bonus. Also those tokens will remain stuck in the Launch manager contract. Following is mint function in launch manager contract

```
function mint(
    address to,
    address referrer,
    uint256 vestingSchedule,
    uint256 minTokensOut,
    uint256[] calldata minShares
) external payable override nonReentrant {
    require(vestingSchedule < _vestingSchedules.length,
LaunchManager__InvalidVestingSchedule());

    if (referrer != ZERO_ADDRESS) {
        require(referrer != to,
LaunchManager__SelfReferralNotAllowed());
        require(referrer != msg.sender,
LaunchManager__SelfReferralNotAllowed());
    }

    VestingSchedule storage schedule =
_vestingSchedules[vestingSchedule];

    require(mintPhaseEndTimestamp > 0 && block.timestamp <
mintPhaseEndTimestamp, LaunchManager__NotInMintPhase());
    require(minShares.length == schedule.tiers.length,
LaunchManager__InvalidVestingSchedule());

    (uint256 tokensToMint, uint256 ethToUse) =
        _calculateMintAmounts(msg.value,
_getCurrentTokenPrice(schedule.tokenPrice), schedule.totalTokens);

    require(tokensToMint > 0,
LaunchManager__VestingScheduleTotalTokensExceeded());
    require(tokensToMint >= minTokensOut,
LaunchManager__InsufficientTokensOut());

    schedule.totalTokens -= tokensToMint;

    (uint256 referralBonus, uint256 minterBonus) =
        _processMintAllocation(to, referrer, tokensToMint, schedule,
minShares);

    _distributeMintEth(ethToUse);

    if (msg.value > ethToUse) {
        (bool success,) = payable(msg.sender).call{value: msg.value -
ethToUse}("");
    }
```

```

        require(success, LaunchManager__EthRefundFailed());
    }

    emit OlympusXMinted(
        to, tokensToMint, referrer, referralBonus, minterBonus,
        vestingSchedule, ethPriceFeed.getLatestEthPriceUSD()
    );
}

```

We can clearly see that it checks that referrer is not to address or msg.sender but it never checks that whether referrer is set to address.this i.e as the launch manager. Now any user can set referrer as address.this . Lets see what is the consequence of this by looking at process mint allocation function

```

/// @dev Handles referral bonuses, liquid token allocation, transfers, and
vesting stake creation
function _processMintAllocation(
    address to,
    address referrer,
    uint256 tokensToMint,
    VestingSchedule storage schedule,
    uint256[] calldata minShares
) internal returns (uint256 referralBonus, uint256 minterBonus) {
    if (referrer != ZERO_ADDRESS) {
        referralBonus =
            (tokensToMint * _referralBonusBps) / BPS;
        minterBonus = referralBonus;

        uint256 totalBonusNeeded = referralBonus + minterBonus;
        if (totalBonusNeeded > referralPoolAllocation) {
            uint256 availableBonus = referralPoolAllocation;
            minterBonus = availableBonus / 2;
            referralBonus = availableBonus - minterBonus;
        }
        referralPoolAllocation -= (referralBonus + minterBonus);
        IERC20(address(olympusX)).transfer(referrer, referralBonus);
    }

    REST OF THE CODE.....
}

```

Now we see that the above logic calculates referrer and minter bonus and then subtracts that from referralPoolAllocation and transfers the referrer the referral bonus. Now in case referrer is set to address.this the referral bonus will be deducted from the referralPoolAllocation but the transfer call will be kind of useless in this case as the tokens remain in the contract therefore ultimately reducing pool allocation. Now these tokens are stuck in this contract and cannot be used for auction.

Recommendations

Also prevent setting referrer as the launch manager address in the mint function.

[M-01] Dec 2025 M-07 fix is bypassable — 1 wei deposit per cycle permanently freezes the auction

Severity

Impact:

High

Likelihood:

Medium

Description

Dec 2025 audit M-07 identified that `totalReservedTokens` in `OlympusXAuction` accumulates permanently when a cycle has zero depositors, eventually bricking `startDistribution()`. The fix added `_releaseUnreservedTokensFromFailedAuctions()`, which releases reserved tokens for cycles where `totalEthDeposited == 0`:

```
function _releaseUnreservedTokensFromFailedAuctions() internal {
    if (currentCycle == 0) return;
    uint256 totalEth = totalEthDeposited[currentCycle];
    uint256 distributed = olympusXDistributedPerCycle[currentCycle];
    if (totalEth == 0 && distributed > 0) {
        totalReservedTokens -= distributed;
    }
}
```

This fix is trivially bypassable. An attacker deposits **1 wei of ETH** during the auction window. This sets `totalEthDeposited[cycle] = 1`, causing the release function's `if (totalEth == 0)` check to fail. The attacker never calls `claim()`. The entire cycle's reserved OLY remains permanently locked in `totalReservedTokens`.

The attack sequence per cycle:

1. Owner calls `startDistribution(bps)` → `totalReservedTokens += amountToDistribute`
2. Attacker calls `deposit{value: 1 wei}()` → `totalEthDeposited[cycle] = 1`
3. Auction ends. Attacker never calls `claim()`
4. Next cycle: `_releaseUnreservedTokensFromFailedAuctions()` checks `totalEthDeposited[currentCycle]` — it's 1, not 0 → **no release**
5. The full `amountToDistribute` from that cycle is permanently locked

`totalReservedTokens` only decrements via `claim()`. With no claim, it only grows. When `totalReservedTokens == poolBalance`:

```

uint256 amountToDistribute = ((poolBalance - totalReservedTokens) *
uint256(distributionBps)) / BPS;
if (amountToDistribute == 0) {
    revert OlympusXAuction__ZeroAmount(); // permanent revert
}

```

Cost to the attacker: 1 wei per cycle. At 100% distribution, a single 1-wei deposit freezes the entire remaining pool in one cycle. There is no admin recovery function, no claim deadline, and `skipCycle()` only advances the timestamp without releasing reserves.

Recommendations

Add an admin-callable function to release reserves from cycles past a configurable unclaim deadline:

```

uint40 public constant CLAIM_GRACE_PERIOD = 730 days; // 2 years

function releaseAbandonedReserves(uint256 cycle) external onlyOwner {
    require(cycle < currentCycle, "OlympusXAuction: cycle not ended");
    require(
        block.timestamp > auctionEndTime + CLAIM_GRACE_PERIOD
        || (cycle < currentCycle - 1),
        "OlympusXAuction: grace period not elapsed"
    );

    uint256 distributed = olympusXDistributedPerCycle[cycle];
    uint256 claimed = totalClaimedPerCycle[cycle];
    uint256 leftover = distributed - claimed;
    if (leftover == 0) return;

    // Freeze the cycle - remaining claimers get 0
    olympusXDistributedPerCycle[cycle] = claimed;
    totalReservedTokens -= leftover;

    emit AbandonedReservesReleased(cycle, leftover);
}

```

[M-02] Vault migration permanently locks all unclaimed inflation rewards

Severity

Impact:

High

Likelihood:

Low

Description

`OlympusX.setStakingVault()` unconditionally removes the old vault from the minting whitelist:

```
function setStakingVault(address _stakingVault) external onlyOwner {
    isWhitelisted[stakingVault] = false;    // old vault loses mint rights
    stakingVault = _stakingVault;
    isWhitelisted[_stakingVault] = true;
}
```

Any user who has accumulated `userUnlockedInflation` in the old vault can never claim it.

`claimInflationRewards()` calls `olympusX.mint(msg.sender, unlocked)` where `msg.sender` is the old vault contract - which is now de-whitelisted, causing a permanent `OlympusX__NotWhitelisted()` revert.

The `userUnlockedInflation` state is not corrupted (the revert rolls back the zero-clear), but it can never be spent. All accumulated inflation for every user in the old vault is permanently inaccessible, with no admin recovery path.

Recommendations

Before removing the old vault from the whitelist, enforce that all users have claimed inflation. Options include adding a migration window that keeps the old vault's minting capability for N days after `setStakingVault` or adding a `migrateInflation(address oldVault, address[] users)` function on the new vault that pulls and re-credits outstanding inflation.

[M-03] Limit-order sell tax is bypassable via `kill()` and arbitrageable via fixed 4% rate

Severity

Impact:

Medium

Likelihood:

Medium

Description

The limit-order tax system has two compounding issues:

1. `kill()` bypasses the tax entirely (0% instead of 4%):

`withdraw()` deducts `LIMIT_ORDER_TAX_BPS = 400` (4%) from ETH proceeds for sell limit orders. `kill()` → `handleKill()` sends the full `epochTotal0` ETH with zero tax deduction:

```
// handleKill - no tax
if (removingAllLiquidity) {
    if (epochTotal0 > 0) key.currency0.take(poolManager, to, epochTotal0,
false);
}
```

`kill()` has no check for `epochInfo.filled`, so it is callable even on fully filled epochs. A sole-LP user (trivially achievable by placing at an empty tick) can always call `kill()` instead of `withdraw()` to receive full ETH proceeds tax-free.

2. Even `withdraw()` charges a fixed 4% vs the dynamic 8-38% market sell tax:

Users who cannot use `kill()` (multi-LP epochs) still benefit from the fixed `LIMIT_ORDER_TAX_BPS = 400` (4%) vs the dynamic 8-38% market sell tax. A user can place a sell limit order one tick below market, trigger a minimal buy from another account to fill it, then `withdraw()` at 4%. At TIER2 (\$25M-\$100M market cap, 28% dynamic tax), the arbitrage saves 24 percentage points per trade.

Combined, sophisticated sellers will always route through limit orders, either paying 0% via `kill()` or 4% via `withdraw()`, undermining the dynamic sell tax as the protocol's primary revenue mechanism.

Recommendations

Apply the same dynamic market-cap-based tax to both `handleKill` and `withdraw` for sell limit order proceeds.

[M-04] `setDistributionBps` doesn't validate mint path - underflow bricks entire mint phase

Severity

Impact:

High

Likelihood:

Low

Description

`buyAndBurnMintBps = 1800` is hardcoded with no setter. `setDistributionBps` validates the five-way BPS sum against `buyAndBurnBps` (e.g., 1600), but `distributeEthFromMint` substitutes `buyAndBurnMintBps` (1800):

```
uint256 toGenesis = total - (toStakers + toBuyAndBurn + toValidators + toFarms);
```

The validation in `setDistributionBps` checks:

```
require(_stakersBps + _buyAndBurnBps + _validatorsBps + _farmsBps + _genesisBps == 10000);  
// Uses _buyAndBurnBps (e.g. 1600), NOT buyAndBurnMintBps (1800)
```

Any configuration where `stakersBps + 1800 + validatorsBps + farmsBps > 10000` causes the `toGenesis` subtraction to underflow. For example, `setDistributionBps(1700, 1600, 5000, 1600, 100)` passes validation (sum = 10000), but `distributeEthFromMint` computes an effective sum of 10100, causing a permanent underflow revert on every `mint()` call until the BPS are reconfigured. The mint phase timer continues ticking during the outage - tokens may go permanently unminted.

Recommendations

In `setDistributionBps`, additionally validate against `buyAndBurnMintBps`: require that `_stakersBps + buyAndBurnMintBps + _validatorsBps + _farmsBps <= BPS`.

[L-01] Swagger UI Falls Back to Hardcoded Credentials in Non-Production

Description

`emergencyWithdrawWstEth` transfers the entire wstETH balance to `StakingVault` but leaves `totalWstETHStaked` and `totalWstETHPrincipal` at their pre-drain values. After the call, every accounting variable that tracks the vault's stake refers to wstETH the vault no longer holds.

ValidatorVault.sol:419-440x:

```
function emergencyWithdrawWstEth() external onlyOwner nonReentrant {  
    _claimFeesFromPoolManager();  
    uint256 ethAmount = pendingEth;  
    uint256 wstEthBalance = IERC20(wstEth).balanceOf(address(this));  
  
    if (ethAmount == 0 && wstEthBalance == 0) revert  
    ValidatorVault__ZeroAmount();  
  
    if (ethAmount > 0) {  
        if (address(this).balance < ethAmount) ethAmount =  
address(this).balance;  
        pendingEth = 0; // <-- ETH side  
reset  
        IStakingVault(stakingVault).distributeRewards{value: ethAmount}
```

```

(address(0), ethAmount);
}

if (wstEthBalance > 0) {
    IERC20(wstEth).approve(stakingVault, wstEthBalance);
    IStakingVault(stakingVault).distributeRewards(wstEth,
wstEthBalance);
    // totalWstETHStaked / totalWstETHPrincipal NOT updated
}

emit EmergencyWithdraw(wstEthBalance, ethAmount);
}

```

`pendingEth` is correctly zeroed, but no equivalent reset exists for the wstETH-side counters. Compare to `harvest`, which explicitly decrements `totalWstETHStaked` for distributed amounts (ValidatorVault.sol:320-322).

Every code path that reads `totalWstETHStaked` returns or operates on a phantom value:

`getWstEthValue` (ValidatorVault.sol:408-411):

```

function getWstEthValue() public view returns (uint256) {
    if (totalWstETHStaked == 0) return 0;
    return IWstETH(wstEth).getStETHByWstETH(totalWstETHStaked);
}

```

After the drain this returns `getStETHByWstETH(stale_total)` — a non-zero ETH value computed against shares the vault does not hold. `getPendingRewards` and `getPendingRewardsWstETH` inherit the staleness, so off-chain dashboards and `StakingVaultLens` consumers see an unchanged AUM after a full drain.

If the protocol resumes after an emergency and `stakeETH` is called:

```

totalWstETHStaked = totalWstETHStaked + wstEthReceived;    // stale + new
totalWstETHPrincipal = totalWstETHPrincipal + amount;

```

Numerical example: pre-emergency `totalWstETHStaked = 100`, `totalWstETHPrincipal = 105`. Drain executes (books unchanged). New `stakeETH(1 ether)` arrives, ~0.95 wstETH minted. Resulting state: actual `balanceOf(this) = 0.95`, `totalWstETHStaked = 100.95`, `totalWstETHPrincipal = 106`. `getWstEthValue() ≈ 106.05`; harvest sees ~0.05 ETH of "rewards", attempts to distribute, reverts. The new staker's capital is locked: held as wstETH but undistributable.

Recommendations

Add the missing state reset inside the `wstEthBalance > 0` branch:

[L-02] Fix L-25 only partially applied: `removeRewardToken()` was never implemented despite being marked as resolved

Description

The audit report OlympusX-security-review_2026-02-16 finding **L-25** states:

"addRewardToken has no cap — unbounded gas DoS on core operation" — **Fix:** "Added `MAX_REWARD_TOKENS` cap and implemented `removeRewardToken()` function"

However, deployed code reveals that while the `MAX_REWARD_TOKENS = 12` cap was implemented in `StakingVault.sol`, the `removeRewardToken()` function was never added. The only token management function that exists is `addRewardToken()`:

```
uint256 public constant MAX_REWARD_TOKENS = 12;

function addRewardToken(address _token) external onlyAuthorized {
    if (rewardTokens.length >= MAX_REWARD_TOKENS) revert
    StakingVault__MaxRewardTokensReached();
    if (isRewardToken[_token]) revert StakingVault__TokenAlreadyAdded();
    uint256 index = rewardTokens.length;
    rewardTokens.push(_token);
    isRewardToken[_token] = true;
    emit RewardTokenAdded(_token, index);
}
// removeRewardToken() — DOES NOT EXIST
```

The audit marked it as resolved, but the fix was only partially applied. The consequences are:

Unrecoverable token slots: Once 12 reward tokens are added, no slots can be freed. If a reward token's contract is compromised, deprecated, or becomes worthless (e.g., a stablecoin that depegs), it permanently occupies one of the 12 slots with no mechanism to remove it.

Inconsistency with documentation and known issues: The project's own documentation states: *"If reward token is removed before a farm is closed, the farm may not be closed."* This statement implies the existence of a `removeRewardToken` function, further suggesting the team believed this functionality was in place.

Recommendations

Implement `removeRewardToken()` as the audit intended.

[L-03] `handlePlaceAfter` sends the hook's entire ETH balance to the `place()` caller

Description

When a user places a limit order via `OlympusXHook.place()`, the internal flow calls `OlympusXHookLib.handlePlaceAfter()` which refunds ETH to the caller. The refund sends the hook contract's **entire** ETH balance, not just the surplus from the current operation:

```
// src/libs/OlympusXHookLib.sol:307-313
uint256 ethBalance = address(hook).balance;
if (ethBalance > 0) {
    (bool success,) = payable(owner).call{value: ethBalance}("");
    if (!success) revert OlympusXHookLib__TransferFailed();
}
```

The hook may accumulate ETH through various interactions (direct sends, or dust from V4 settlement). The first `place()` caller after any such accumulation receives this windfall. Additionally, if the caller is a contract without a `receive()` function, the ETH transfer reverts, permanently blocking all future `place()` calls until the hook is killed.

Recommendations

Track only the ETH owed to the current `place()` call using a before/after balance delta

[L-04] Anyone can earn large rewards by staking just 1 wei Olympus token after zero stakers period

Description

When there are no stakers in the staking vault and total shares are equal to dead shares then in this case the rewards are just accounted but the cumulative rewards per share value is not updated. It is only when there are more than dead shares that these rewards are actually distributed to the share holders. So suppose there was zero staker period and some rewards were received by the vault. Now seeing this an attacker just stakes 1 wei olympus token now see how shares are calculated

```
function calculateShares(uint256 amount, uint16 stakeDays, uint256
currentShareRate)
    public
    pure
    returns (uint256 shares)
{
    shares = (amount * PRECISION) / currentShareRate;
    uint256 bonusBPS = Math.mulDiv(
        MAX_BONUS_BPS, stakeDays - MIN_STAKE_DAYS, MAX_STAKE_DAYS -
MIN_STAKE_DAYS, Math.Rounding.Floor
    );
    uint256 bonus = (shares * bonusBPS) / BPS;
```

```
        shares += bonus;
    }
```

PRECISION is $1e24$ Therefore if user stakes 1 wei olympus token shares will be equal to $1 * 1e24/1e18 = 1e6$ shares. Also dead shares equal to 1000 therefore user has quite large amount of shares in comparison to dead shares. Now total shares are greater than dead shares and this user can call claim rewards and will earn way large amount of rewards in comparison to his stake of 1 wei. Not only this it also leads to accumulation of rewards for dead address.

Recommendations

Apply a min stake amount variable as well in `_startstake` function.

[L-05] No functionality to fully withdraw the staked eth

Description

There is no functionality in validator vault function to withdraw staked eth in case of normal use whenever funds are needed. There only exists an emergency function which transfer pending eth and the wrapped st eth balance of the contract to staking vault as rewards. There is only harvest function which reaps rewards earned on top of the currently staked eth.

Recommendations

Introduce a normal function unlike the emergency one to unstake the eth.

[L-06] New observation is written even when fees are collected from the pools

Description

It is clearly written in the comments of `_updatePool` function that it should only be called when there is change in pool price or the liquidity as can be seen below

```
/// @dev Called before any action that potentially modifies pool price or
liquidity, such as swap or modify position
function _updatePool(PoolKey calldata key) private {
    PoolId id = key.toId();
    (, int24 tick,,) = manager.getSlot0(id);

    uint128 liquidity = manager.getLiquidity(id);

    (states[id].index, states[id].cardinality) =
observations[id].write(
    states[id].index, _blockTimestamp(), tick, liquidity,
states[id].cardinality, states[id].cardinalityNext
```

```

    );
}

```

Now when only fees are collected it doesn't modify the pool price of liquidity but it still calls before modify hook which calls `_updatePool` function. Following is from UniV4 hooks library

```

/// @notice calls beforeModifyLiquidity hook if permissioned and validates
return value
function beforeModifyLiquidity(
    IHooks self,
    PoolKey memory key,
    ModifyLiquidityParams memory params,
    bytes calldata hookData
) internal noSelfCall(self) {
    if (params.liquidityDelta > 0 &&
self.hasPermission(BEFORE_ADD_LIQUIDITY_FLAG)) {
        self.callHook(abi.encodeCall(IHooks.beforeAddLiquidity,
(msg.sender, key, params, hookData)));
    } else if (params.liquidityDelta <= 0 &&
self.hasPermission(BEFORE_REMOVE_LIQUIDITY_FLAG)) {
        self.callHook(abi.encodeCall(IHooks.beforeRemoveLiquidity,
(msg.sender, key, params, hookData)));
    }
}

```

It can be clearly seen that when liquidity delta is 0 then before remove liquidity function is called in hook.

```

/// @inheritdoc BaseHook
function _beforeRemoveLiquidity(address, PoolKey calldata key,
ModifyLiquidityParams calldata, bytes calldata)
    internal
    override
    returns (bytes4)
{
    // _beforeModifyPosition for TruncGeoOracle
    _beforeModifyPosition(key);
    return this.beforeRemoveLiquidity.selector;
}

```

and `_beforeModifyPosition` function calls the update pool function

```

function _beforeModifyPosition(PoolKey calldata key) internal {
    _updatePool(key);
}

```

Due to this unnecessary observation is written which only increase the observation array which can even incur way more gas when there is time look up for used for calculation of twap price or for checking missing observation.

Recommendations

In `_beforeRemoveLiquidity` function check that the liquidity delta is strictly negative.

[L-07] Claiming rewards for dead address is possible

Description

In claim rewards for user function it is possible to claim rewards for dead address. It is kind of useless to allow this. It can be info finding as well .

Recommendations

Don't allow dead address to be passed in claim rewards for user.

[L-08] TaxRouter all-or-nothing distribution - single broken recipient bricks all protocol ETH flow

Description

`TaxRouter._distributeEthInternal()` sends ETH to four recipients via `.call{value}` and reverts the entire function if any single send fails:

```
if (toStakers > 0) {
    (success,) = stakingVault.call{value: toStakers}("");
    if (!success) revert TaxRouter__TransferFailed();
}
if (toBuyAndBurn > 0) {
    (success,) = buyAndBurn.call{value: toBuyAndBurn}("");
    if (!success) revert TaxRouter__TransferFailed();
}
// ... same for validatorVault and farmKeeper
```

TaxRouter auto-distributes on every ETH receipt via `receive()`. Any destination whose `receive()` reverts causes a protocol-wide revenue halt. The cascade propagates: TaxRouter reverts → `poolManager.take(taxRouter, taxAmount)` reverts → the entire V4 swap fails. Simultaneously, `OlympusXAuction.deposit()` and `LaunchManager.mint()` both send ETH to TaxRouter and also fail. All sell taxes, mint distributions and auction deposits are blocked atomically until the admin corrects the broken recipient.

Recommendations

Wrap each recipient call in a try/catch pattern, storing failed distributions in a mapping for later retry rather than reverting the entire flow.

[L-09] Max stakes per user limit missing - unbounded `userStakeIds` growth

Description

`_startStake()` pushes to the `userStakeIds` array with no length check:

```
uint256 stakeIndex = userStakeIds[_user].length; // no max check
userStakeIds[_user].push(stakeId);                // unbounded growth
```

While no write function iterates this array, the unbounded growth breaks view functions in `StakingVaultLens.getUserStakesNearingGA()` iterates the full array twice with external calls per element. At ~3000 stakes, gas for this call exceeds the block limit, making frontend and integration visibility unavailable. `LaunchManager.startStakeForUser()` also contributes to growth without the user's control.

Recommendations

Add a `MAX_STAKES_PER_USER` constant (e.g., 50) and enforce it in `_startStake()` with a require check on `userStakeIds[_user].length`.

[L-10] `setWethToken(false)` desyncs from cached `isNativePool` - TaxRouter cascade revert

Description

`setWethToken()` updates the global `isWethToken` mapping but does not update the cached `isNativePool` flag on existing farms:

```
function setWethToken(address _wethToken, bool _isWethToken) external
onlyOwner {
    isWethToken[_wethToken] = _isWethToken; // mapping updated
    // farm.isNativePool NOT updated for existing farms
}
```

When a V3 WETH farm exists and WETH is deregistered, `allocateEthToFarm()` still enters the native pool path because `farm.isNativePool` was cached at creation time. It tries to resolve which token is WETH - both return false after deregistration - and calls `IWETH9(currency1).deposit{value}()` on a non-WETH ERC20, reverting. This cascade propagates to TaxRouter, blocking all protocol ETH flows.

Currently dormant (only V4 native OLY farm deployed), but activates if V3/V4 WETH farms are added.

Recommendations

When `setWethToken()` changes a token's status, iterate existing farms and update their cached `isNativePool` flag. Alternatively, compute `isNativePool` dynamically from the `isWethToken` mapping on each call.

[L-11] `stakeETH` wraps donated stETH via full balance read - phantom harvest rewards

Description

`stakeETH()` reads `stETH.balanceOf(address(this))` after `submit()`, capturing any pre-existing donated stETH in the wrap amount:

```
stETH.submit{value: amount}(address(0));
uint256 stETHReceived = stETH.balanceOf(address(this));    // FULL
balance, not delta
uint256 wstEthReceived = wstETHContract.wrap(stETHReceived); // wraps ALL

totalWstETHStaked = totalWstETHStaked + wstEthReceived;    // inflated
by donation
totalWstETHPrincipal = totalWstETHPrincipal + amount;      // correct
(ETH only)
```

The mismatch between `totalWstETHStaked` (inflated) and `totalWstETHPrincipal` (correct) causes `harvest()` to compute phantom rewards equal to the donated amount's ETH value. The attack is not profitable - the attacker permanently loses their donation - but it distorts reward accounting.

Recommendations

Use a before/after balance pattern: record `stETH.balanceOf(address(this))` before `submit()`, then compute `stETHReceived = balanceAfter - balanceBefore`.

[L-12] `_prepareWithdrawalAmounts` split produces sub-minimum amounts - harvest DoS

Description

When a Lido withdrawal amount requires splitting, `_prepareWithdrawalAmounts()` splits one full-sized request into two halves without checking against the minimum:

```
} else if (needsSplit) {
    uint256 halfMax = _maxWithdrawalAmount / 2;
```

```

    amounts[fullRequests] = halfMax; // can be <
    minWithdrawalAmount
    amounts[fullRequests + 1] = halfMax + remainder;
}

```

Both `setMinWithdrawalAmount` and `setMaxWithdrawalAmount` enforce `min < max` but neither enforces `min <= max / 2`. If `minWithdrawalAmount = 600 ether` and `maxWithdrawalAmount = 1000 ether`, the split produces `[500, 550]` - both below the 600 minimum. Lido's `requestWithdrawals()` rejects sub-minimum amounts, reverting `harvest()`.

Recommendations

Add a validation in both setters ensuring `minWithdrawalAmount <= maxWithdrawalAmount / 2`. Alternatively, check if `halfMax < minWithdrawalAmount` and use a single request instead.

[L-13] `harvest()` bare call to Lido WithdrawalQueue blocks all staker rewards

Description

`harvest()` calls `IWithdrawalQueue.requestWithdrawals()` without try/catch. If Lido's withdrawal queue is paused (has happened historically during protocol upgrades), this call reverts and the entire `harvest()` transaction rolls back - including the staker wstETH distribution that already succeeded earlier in the same function:

```

// Staker distribution succeeds:
IStakingVault(stakingVault).distributeRewards(wstEth, wstETHToStakers);

// Bare call to Lido - no try/catch:
uint256[] memory requestIds =
    IWithdrawalQueue(withdrawalQueueAddr).requestWithdrawals(amounts,
address(this));
// If Lido queue paused → REVERT → staker distribution ROLLED BACK

```

Recommendations

Wrap the `requestWithdrawals()` call in a try/catch. On failure, redirect the BuyAndBurn wstETH portion to the reinvest path so staker distributions are not blocked.

[I-01] `StakingVault._withdrawTokens` Silently Caps ETH Withdrawals to Available Balance

Description

If `pendingWithdrawals` exceeds actual balance (due to rounding accumulation or accounting drift), users' withdrawals are silently capped and their `pendingWithdrawals` is zeroed — they lose the difference permanently.

```
uint256 available = address(this).balance;
if (amount > available) {
    amount = available;
}
pendingWithdrawals[_user][token] = 0;
```

Recommendations

Reduce `pendingWithdrawals` by the actually transferred amount, not the full claimed amount.

[I-02] `LaunchManager.mint` self-referral check is trivially bypassed via sybils

Description

`LaunchManager.mint(address to, address referrer, ...)` awards a bonus drawn from the `referralPoolAllocation`. The self-referral guard at `LaunchManager.sol:218–221` checks only that `referrer != to` and `referrer != msg.sender`; A user holding two EOAs `A` and `B` can call `mint(to = A, referrer = B)` from `A` and collect the full bonus on both sides.

Recommendations

Consider if this is the expected behavior. If not, remove the referral system altogether.

[I-03] Redundant zero `_owner` address check

Description

The constructor in `SwapActions.sol` has redundant checks for zero `_owner` address since the `Ownable` contract already validates the owner address.

```
constructor(
    address _owner,
    // V3
    address _v3SwapRouter,
    // V4
    address _v4Router
)
    Ownable(_owner)
    notZeroAddress(_owner)
    notZeroAddress(_v3SwapRouter)
```

```
        notZeroAddress(_v4Router)
    {
        //
```

Recommendations

It can be safely removed.

[I-04] Calling `extendMintPhase` before `startMintPhase` permanently bricks launch

Description

`extendMintPhase()` adds `MINT_PHASE_EXTENSION_DURATION` to `mintPhaseEndTimestamp`. If called when `mintPhaseEndTimestamp == 0` (before `startMintPhase`), the timestamp becomes a non-zero value. `startMintPhase()` requires `mintPhaseEndTimestamp == 0` and no function ever resets it, making launch permanently impossible.

Recommendations

Add `require(mintPhaseEndTimestamp > 0)` to `extendMintPhase()` or use the existing `require(block.timestamp < mintPhaseEndTimestamp)` pattern to ensure mint phase is already active.

[I-05] `createAndInitializeLiquidityPool` is permissionless with oracle-dependent minting

Description

`LaunchManager.createAndInitializeLiquidityPool()` has no access control. The amount of OLY minted for the initial liquidity pool depends on `ethPriceFeed.getLatestEthPriceUSD()` at call time. A griever could call this function during an oracle price spike to mint fewer OLY tokens, resulting in a less favorable initial pool ratio.

Recommendations

Add `onlyOwner` access control or use a TWAP for the oracle price used in pool initialization.

[I-06] Grace period effective window is ~8 days, not the documented 7

Description

`GracePeriodPenalties.calculateGracePeriodPenalties()` uses `daysLate = (currentTimestamp - graceEndTime) / 1 days`. Floor division means the first 86,399 seconds (~24 hours) after the grace period ends yield `daysLate = 0`, giving users an extra penalty-free day beyond the intended 7-day window.

Recommendations

If the 7-day window is intended to be exact, use `daysLate = ((currentTimestamp - graceEndTime) + 1 days - 1) / 1 days` (ceiling division). Otherwise, document the effective ~8-day behavior.

[I-07] `MarketCapLib.getTokenPriceInETH` has misleading function name and comments

Description

The function is named `getTokenPriceInETH` and the comment at line 61 says "OlympusX price in ETH," implying it returns how much ETH per 1 OLY. It actually returns `token1/token0` = OLY per ETH (how many OLY tokens you get for 1 ETH). The downstream math in `_getTokenPriceInUsd (ethPriceUSD / tokenPriceInETH)` is correct only because the value is OLY-per-ETH, not ETH-per-OLY. A developer reading the function name would assume the inverse and use it with multiplication instead of division, introducing a critical pricing bug.

Recommendations

Rename to `getOlympusXPerETH()` or correct the comment at line 61 to "OlympusX per ETH (how many OlympusX tokens per 1 ETH)."

[I-08] Duplicate `ETHReceived` event emission in `_claimFeesFromPoolManager`

Description

When ERC6909 fees are claimed from the pool manager, two identical `ETHReceived` events are emitted for a single ETH movement:

```
// Event #1 - during unlock callback via receive():
receive() external payable {
    unchecked { pendingEth = pendingEth + msg.value; }
    emit ETHReceived(msg.sender, msg.value);
}

// Event #2 - explicitly after unlock returns:
poolManager.unlock(abi.encodeCall(this.unlockCallbackClaimFees,
```

```
(balance)));  
emit ETHReceived(address(poolManager), balance);
```

Off-chain indexers counting `ETHReceived` events will double-count ERC6909 fee claims.

Recommendations

Remove the explicit `emit ETHReceived` from `_claimFeesFromPoolManager()`. The `receive()` event already covers this and `FeesClaimedFromPoolManager` provides fee-specific context.

[I-09] Multiple critical admin setters across five contracts emit no events

Description

Several state-changing admin functions that modify protocol-critical parameters emit no events, making configuration changes invisible to off-chain monitoring. Each contract has other setters that DO emit events, confirming this is an inconsistency rather than a design choice.

Affected functions:

1. `TaxRouter.setDistributionBps()` - changes the 5-way revenue split with no event. All 5 address setters in the same contract emit `AddressUpdated`.
2. `FarmKeeper: setLaunchManager(), setStakingVault(), setBuyAndBurn(), setOlympusXHook()` - 4 address setters with zero events. Other setters in the same contract emit events.
3. `EthPrice.setPriceConfig()` - overwrites the entire oracle configuration (Chainlink feed, stale threshold, TWAP fallback) with no event. The contract has zero event definitions.
4. `OlympusXHook.setTaxEnabled()` - toggles the global sell tax system. Any whitelisted address can call it with no on-chain trace. Every other setter in OlympusXHook emits an event.
5. `OlympusXHook.setTaxMode()` - switches between Dynamic (8-38%) and Fixed tax modes, directly controlling the protocol's primary revenue rate. Every other setter in OlympusXHook emits an event.

Recommendations

Add event emissions to all affected functions. For `setTaxEnabled`, include `msg.sender` in the event since multiple addresses may have whitelist access.

[I-10] `FarmKeeperLib.validateSwapSlippage` allows 100% slippage

Description

`validateSwapSlippage()` uses `slippage > 100_00` (strictly greater than), allowing `slippage = 10000` (100%). In `SwapActionsLib._calculateAmountOutMinimum()`, `BPS - slippageBPS = 0`, producing `amountOutMinimum = 0` - zero slippage protection. This is asymmetric with `validateLiquiditySlippage` which caps at 5000 (50%).

Recommendations

Change to `slippage >= BPS` or set a reasonable upper bound like 5000 to match liquidity slippage validation.

[I-11] `FarmKeeper.setLiquiditySlippage` natspec does not match code

Description

The natspec for `setLiquiditySlippage` documents the valid range as `(1-2500)`, but `FarmKeeperLib.validateLiquiditySlippage()` enforces `(1-5000)`.

Recommendations

Update the natspec to reflect the actual enforced range of 1-5000.

[I-12] Permissionless `claimRewardsForUser` enables reward timing manipulation

Description

`claimRewardsForUser()` has no access control on the `_user` parameter - any address can force a claim for any staker:

```
function claimRewardsForUser(address _user, address _token)
    external nonReentrant
    returns (uint256[5] memory claimedPerCycle)
{
    return _claimRewardForToken(_user, _token); // no restriction on _user
}
```

This function triggers pending cycle payouts before claiming. An attacker can front-run a victim's share-increasing transaction (e.g., `startStake`) with `claimRewardsForUser(victim, token)`, causing the payout to trigger at the victim's current (lower) share count. No funds are stolen - the impact is a minor redistribution from the victim to the pool.

Recommendations

Add `require(msg.sender == _user)` or document the permissionless design as intentional for keeper-driven claims.

[I-13] Dead code in `_executeClaim` wastes gas on unused external reads

Description

`_executeClaim()` in `StakingVaultLens` reads `pendingWithdrawals` (for ETH) or `balanceOf` (for ERC20) into local variables before calling `claimRewardsForUser()`, but neither variable is ever referenced afterward:

```
uint256 balanceBefore;
uint256 pendingEthBefore;

if (_token == address(0)) {
    pendingEthBefore = stakingVault.pendingWithdrawals(_user, address(0));
    // never used
} else {
    balanceBefore = IERC20(_token).balanceOf(_user);
    // never used
}

uint256[5] memory claimedPerCycle = stakingVault.claimRewardsForUser(_user,
_token);
// balanceBefore and pendingEthBefore absent from this point
```

Recommendations

Remove the `balanceBefore`, `pendingEthBefore` variables and the associated `if/else` block.